

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20.1

MANAGING TOKENS

 Copy as Markdown

Managing tokens

An authentication token is the credential the app sends with each request to prove who the signed-in user is. Token managers issue these tokens when a user signs in, and validate, revoke, and list them afterwards.

Default token managers

Serverpod provides two built-in token managers:

- `JwtTokenManager` for JWT-based authentication. See [JWT token manager](#) for details.
- `ServerSideSessionsTokenManager` for session-based authentication. See [Server-side sessions token manager](#) for details.

The main trade-off between them:

	<code>JwtTokenManager</code>	<code>ServerSideSessionsTokenManager</code>
Validation	Stateless, no database query	Database query per validation
Revocation	Takes effect when the short-lived access token expires	Immediate

Pick JWT to avoid database load on every request. Pick server-side sessions when revocation must take effect immediately.

Using the token managers



After you configure at least one token manager with `pod.initializeAuthServices()` (see [Setup](#)), access the token manager through the `AuthServices.instance.tokenManager` property.

```
final tokenManager = AuthServices.instance.tokenManager;
```

The property returns a `MultiTokenManager`, a wrapper that combines all configured token managers. Serverpod creates it automatically when you initialize the authentication services. The first builder you pass becomes the primary token manager. The `MultiTokenManager`:

- Uses the primary token manager for issuing new tokens.
- Validates tokens against all managers (primary and additional).
- Delegates management operations to all managers.

Token validation flow

When validating a token, the `MultiTokenManager`:

1. Tries the primary token manager first.
2. If validation fails, tries each additional token manager in order.
3. Returns the first successful validation result.
4. Returns `null` if all managers fail to validate.

This allows you to support multiple token types simultaneously, which is useful for:

- Migrating from one token type to another.
- Supporting legacy tokens alongside new tokens.
- Using different token types for different use cases.

Token lifecycle management

Issuing tokens

Identity providers issue tokens automatically when users authenticate. For your own sign-in flow, call `issueToken`:

```
final authSuccess = await AuthServices.instance.tokenManager.issueToken(  
  session,  
  authUserId: userId,  
  method: 'custom',  
  scopes: {Scope.admin},  
);
```

Return the `AuthSuccess` to the app. It carries the token, its expiry, the granted scopes, and, for JWT, the refresh token. The `issueToken` method is built for sign-in:

- If the caller is already signed in as a different user, it throws a `SignInWhileAuthenticatedException`.
- On a web request with [cookie-based authentication](#), it sends the refresh token (JWT) or the session token (server-side sessions) as an `httpOnly` cookie instead of in the response body.

To create a token for another user, or a token the caller must receive in the response body, call `createToken` instead. For example, use it in an admin flow or for a personal access token. It skips the sign-in check and the cookie delivery, so the returned `AuthSuccess` always includes the token and, for JWT, the refresh token:

```
final authSuccess = await AuthServices.instance.tokenManager.createToken(  
    session,  
    authUserId: targetUserId,  
    method: 'pat',  
);
```

Attaching metadata to tokens

You can attach metadata to tokens in two ways. Configure a global callback on the token manager, or insert a metadata row right after issuing the token. For more details, see the specific configuration sections for [server-side sessions](#) and [JWT](#).

Validating tokens

Tokens are validated automatically by the authentication handler, the hook Serverpod runs for every request that carries an authentication token. You can also validate tokens manually:

```
final authInfo = await AuthServices.instance.tokenManager.validateToken(  
    session,  
    token,  
);  
  
if (authInfo != null) {  
    // Token is valid  
    final authUserId = authInfo.authUserId;  
} else {  
    // Token is invalid or expired  
}
```

Revoking tokens

Revoke specific tokens by token ID:

```
await AuthServices.instance.tokenManager.revokeToken(  
    token,
```

```
session,  
tokenId: 'token-id-here',  
);
```

If you attach custom metadata to [server-side sessions](#) or [JWT tokens](#), you can look up token IDs in your metadata tables, for example by device or user agent. Pass those IDs to `revokeToken` to revoke exactly those tokens.

Revoke all tokens for a user:

```
await AuthServices.instance.tokenManager.revokeAllTokens(  
  session,  
  authUserId: userId,  
);
```

Revoke tokens by method:

```
await AuthServices.instance.tokenManager.revokeAllTokens(  
  session,  
  authUserId: userId,  
  method: 'email',  
);
```

Listing tokens

List all tokens for a user:

```
final tokens = await AuthServices.instance.tokenManager.listTokens(  
  session,  
  authUserId: userId,  
);  
  
for (final token in tokens) {  
  print('Token: ${token.tokenId}, Method: ${token.method}');  
}
```

List tokens by method (for example, `'google'`, `'email'`, or `'apple'`):

```
final tokens = await AuthServices.instance.tokenManager.listTokens(  
  session,  
  authUserId: userId,  
  method: 'google',  
);
```

Accessing specific token managers

If you configure more than one token manager, retrieve a specific one with the static `AuthServices.getTokenManager<T>()` method.

```
final jwtManager = AuthServices.getTokenManager<JwtTokenManager>();  
final sessionManager = AuthServices.getTokenManager<ServerSideSessionsTokenManager>();
```

Related

- [Setup](#): configure token managers with `initializeAuthServices`.
- [JWT token manager](#): stateless tokens with automatic refresh.
- [Server-side sessions token manager](#): database-backed sessions with immediate revocation.